



Introduction to **Information Retrieval**

Mustafa Jarrar



Watch this lecture and download the slides



Course Page: <http://www.jarrar.info/courses/NLP/>

More Online Courses at: <http://www.jarrar.info>

Acknowledgement: This lecture is largely based on Chris Manning online course on NLP, which can be accessed at <http://www.youtube.com/watch?v=s3kKIUBa3b0>

Keywords: Natural Language Processing, Information Retrieval, Precision, Recall, Inverted Index , Positional Indexes, Query processing, Merge Algorithm, Biwords, Phrase queries Qabas, Sinalab, birzeit_university, machine learning, arabicnlp, aclproc, معالجة ألية للغت الطبيعية , تطبيقات لغوية , استعمال , استرجاع المعلومات , الدقة , استرجاع المعلومات , استرجاع المعلومات , استرجاع المعلومات

Introduction to **Information Retrieval**

In this lecture:



- Part 1: **Information Retrieval Basics**
- ☐ Part 2: Term-document incidence matrices
- ☐ Part 3: Inverted Index
- ☐ Part 4: Query processing with inverted index
- ☐ Part 5: Phrase queries and positional indexes
- ☒ Part 6: Discussion
- ☐ Part 7: Practice

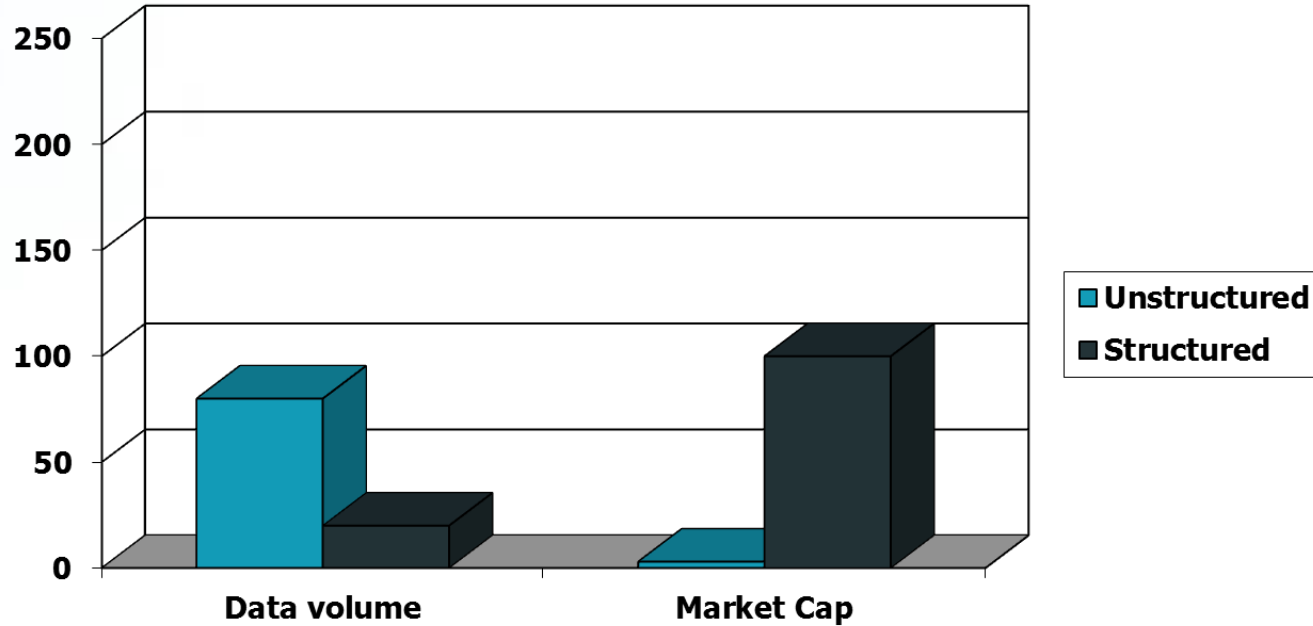
Information Retrieval

Information Retrieval (IR) is finding material (usually documents) of an unstructured nature (usually text) that satisfies an information need from within large collections (usually stored on computers).

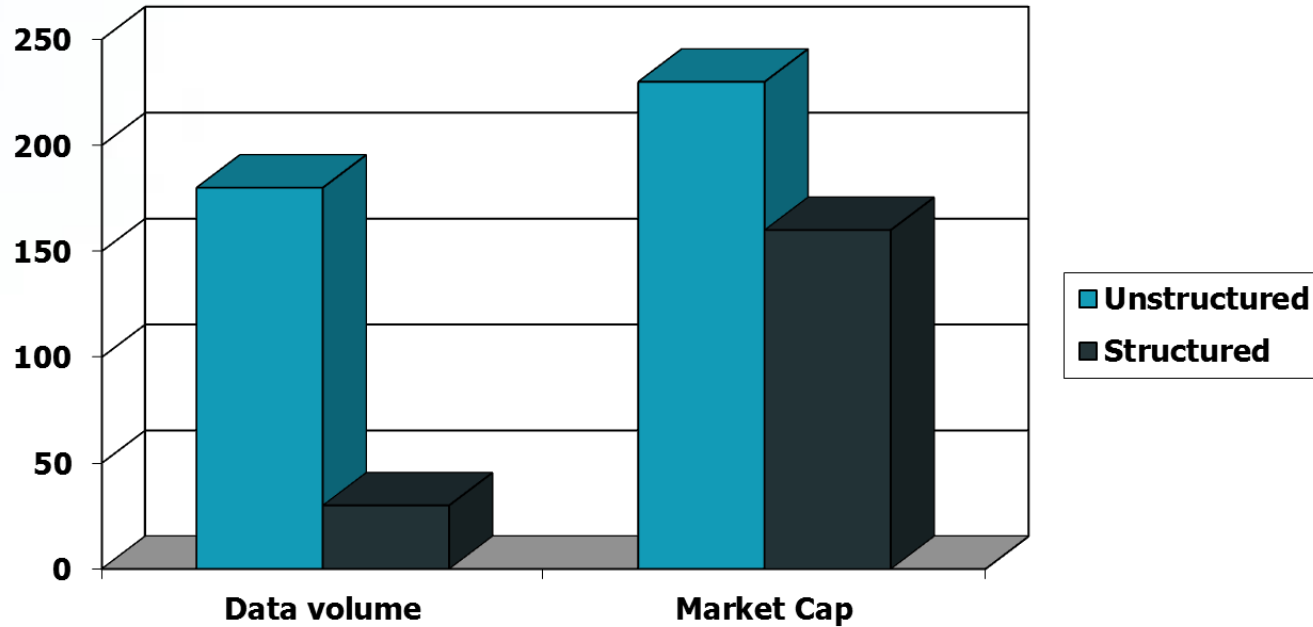
These days we frequently think first of **web search**, but there are many other cases:

- E-mail search
- Searching your laptop
- Corporate knowledge bases
- Legal information retrieval

Unstructured (text) vs. structured (database) data in the mid-nineties



Unstructured (text) vs. structured (database) data later



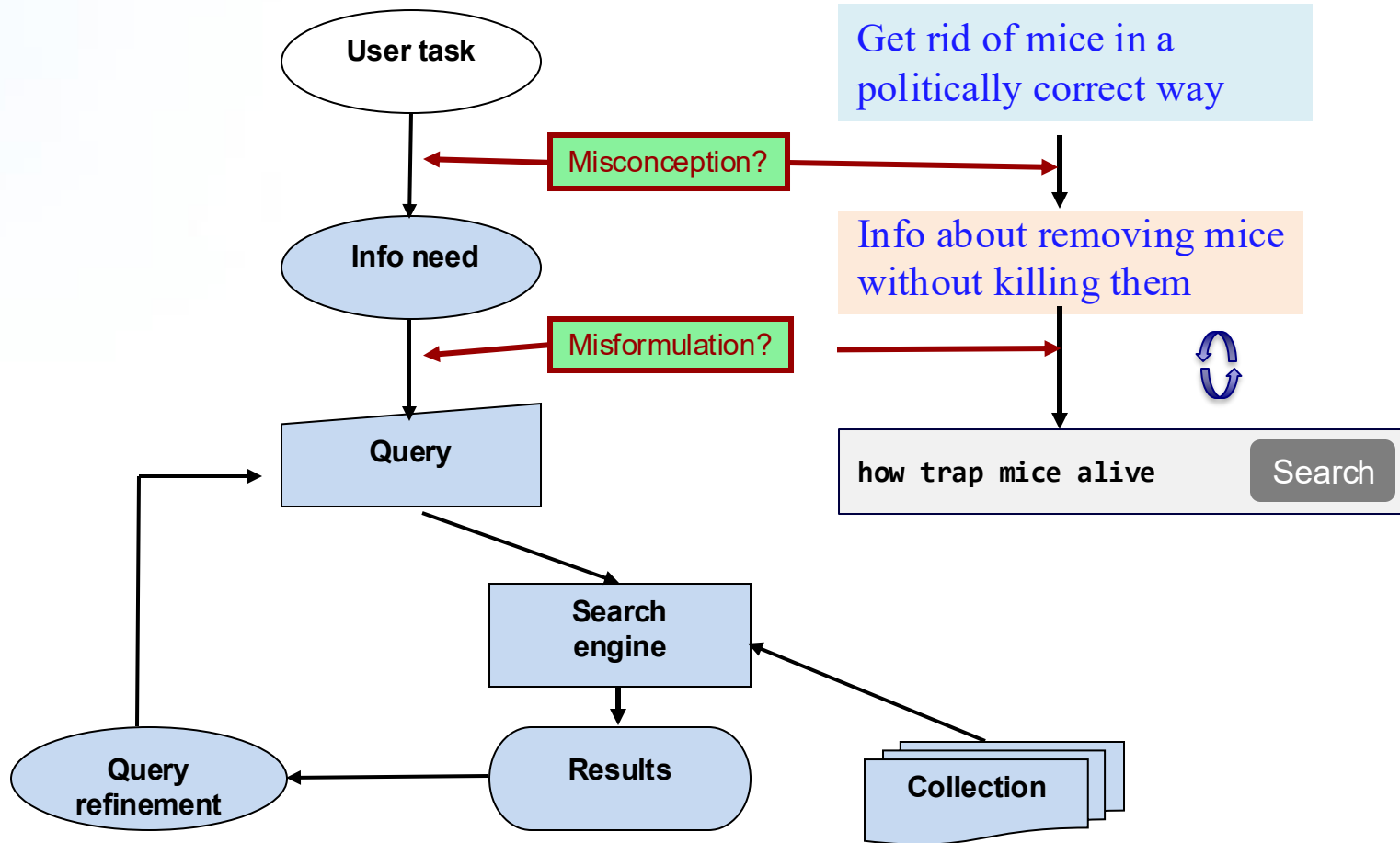
Basic Assumptions of Information Retrieval

Collection: A set of documents

Assume it is a static collection (but in other scenarios we may need to add and delete documents).

Goal: Retrieve documents with information that is **relevant** to the user's **information need** and helps the user complete a **task**.

Classic Search Model



How good are the retrieved docs?

- **Precision:** Fraction of retrieved docs that are relevant to the user's information need

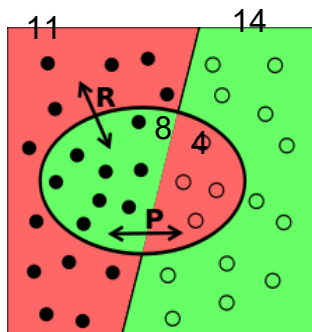
نسبة الصحيح من بين ما تم استرجاعه

$$\text{precision} = \frac{|\{\text{relevant documents}\} \cap \{\text{retrieved documents}\}|}{|\{\text{retrieved documents}\}|}$$

- **Recall:** Fraction of relevant docs in collection that are retrieved

نسبة الصحيح المسترجع من بين جميع الصحيح

$$\text{recall} = \frac{|\{\text{relevant documents}\} \cap \{\text{retrieved documents}\}|}{|\{\text{relevant documents}\}|}$$



In this figure the relevant items are to the left of the straight line while the retrieved items are within the oval. The red regions represent errors. On the left these are the relevant items not retrieved (false negatives), while on the right they are the retrieved items that are not relevant (false positives).

$$P = 8/12 = 66\%, \quad R = 8/14 = 57\%$$

Are these measures always useful as in case of huge collections?
Ranking becomes more important sometimes.

Introduction to Information Retrieval

In this lecture:

☐ Part 1: Information Retrieval Basics

 Part 2: **Term-document incidence matrices**

☐ Part 3: Inverted Index

☐ Part 4: Query processing with inverted index

☐ Part 5: Phrase queries and positional indexes

☒ Part 6: Discussion

☒ Part 7: Practice

Indexing Unstructured Data

Which plays of Shakespeare contain the words ***Brutus AND Caesar*** but ***NOT Calpurnia***?

One could grep all of Shakespeare's plays for ***Brutus*** and ***Caesar***, then strip out lines containing ***Calpurnia***?

Why is that not the answer?

- Slow (for large corpora)

- NOT Calpurnia*** is non-trivial

- Other operations (e.g., find the word ***Romans*** near ***countrymen***) not feasible

- Ranked retrieval (best documents to return)

Term-Document Incidence Matrices

Brutus AND Caesar BUT NOT Calpurnia

	Antony and Cleopatra	Julius Caesar	The Tempest	Hamlet	Othello	Macbeth
Antony	1	1	0	0	0	1
Brutus	1	1	0	1	0	0
Caesar	1	1	0	1	1	1
Calpurnia	0	1	0	0	0	0
Cleopatra	1	0	0	0	0	0
mercy	1	0	1	1	1	1
worser	1	0	1	1	1	0

1 if play contains word,
0 otherwise

Incidence Vectors

So we have a 0/1 vector for each term.

To answer query: take the vectors for *Brutus*, *Caesar* and *Calpurnia* (complemented) → bitwise *AND*.

110100 *AND*

110111 *AND*

101111 =

100100

	 Antony and Cleopatra	Julius Caesar	The Tempest	 Hamlet	Othello	Macbeth
Antony	1	1	0	0	0	1
Brutus	1	1	0	1	0	0
Caesar	1	1	0	1	1	1
Calpurnia	0	1	0	0	0	0
Cleopatra	1	0	0	0	0	0
mercy	1	0	1	1	1	1
worser	1	0	1	1	1	0

Answers to Query



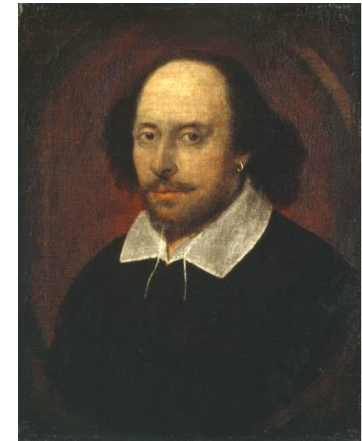
Antony and Cleopatra, Act III, Scene ii

Agrippa [Aside to DOMITIUS ENOBARBUS]: Why, Enobarbus,
When Antony found Julius **Caesar** dead,
He cried almost to roaring; and he wept
When at Philippi he found **Brutus** slain.



Hamlet, Act III, Scene ii

Lord Polonius: I did enact Julius **Caesar** I was killed i' the
Capitol; **Brutus** killed me.



With Bigger Collections!

Consider $N = 1$ million documents, each with about 1000 words.

Avg 6 bytes/word including spaces/punctuation
= 6GB of data in the documents.

Say there are $M = 500\text{K}$ *distinct* terms among these.

We Can't Build this Matrix!

500K x 1M matrix has half-a-trillion 0's and 1's.

But it has no more than one billion 1's. (=1000 words x 1 M doc.)

→ matrix is extremely sparse.

What's a better representation?

We only record the 1 positions.

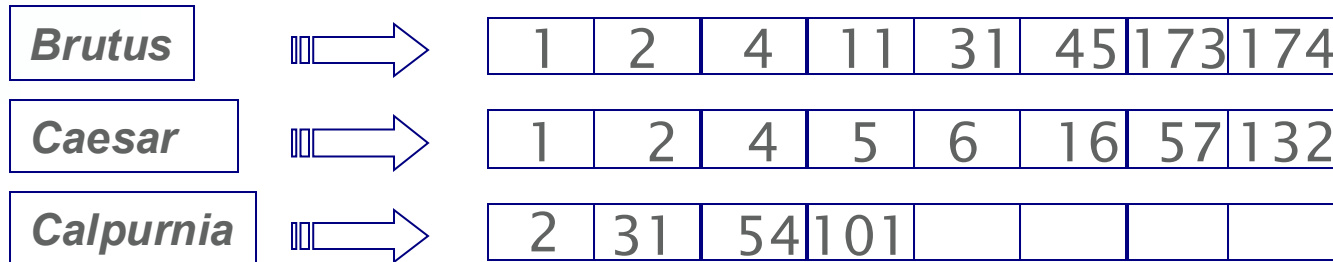
Introduction to Information Retrieval

In this lecture:

- ☐ Part 1: Information Retrieval Basics
- ☐ Part 2: Term-document incidence matrices
-  ☐ Part 3: **Inverted Index**
- ☐ Part 4: Query processing with inverted index
- ☐ Part 5: Phrase queries and positional indexes
- ☐ Part 5: Discussion
- ☐ Part 7: Practice

What is an Inverted Index?

- For each term t , we must store a list of all documents that contain t .
Identify each doc by a **docID**, a document serial number
- Can we use fixed-size arrays for this?



What happens if the word **Caesar** is **added** to document 14?

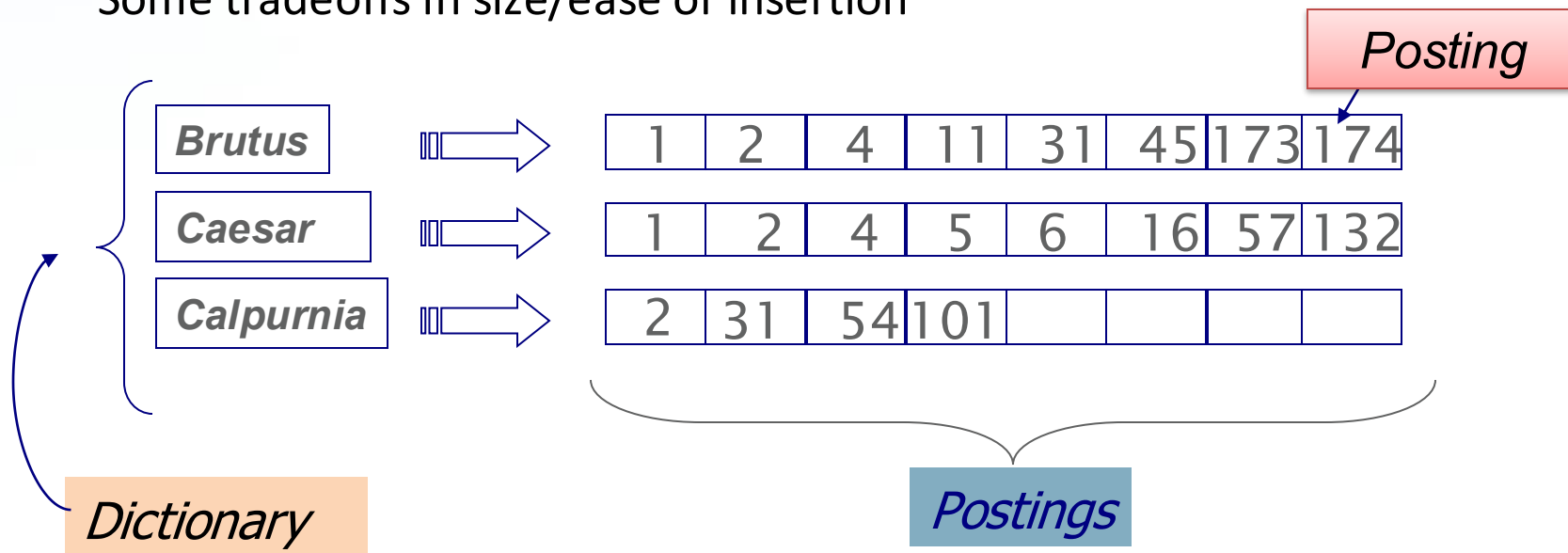
Inverted index

We need variable-size **postings lists**

On disk, a continuous run of postings is normal and best

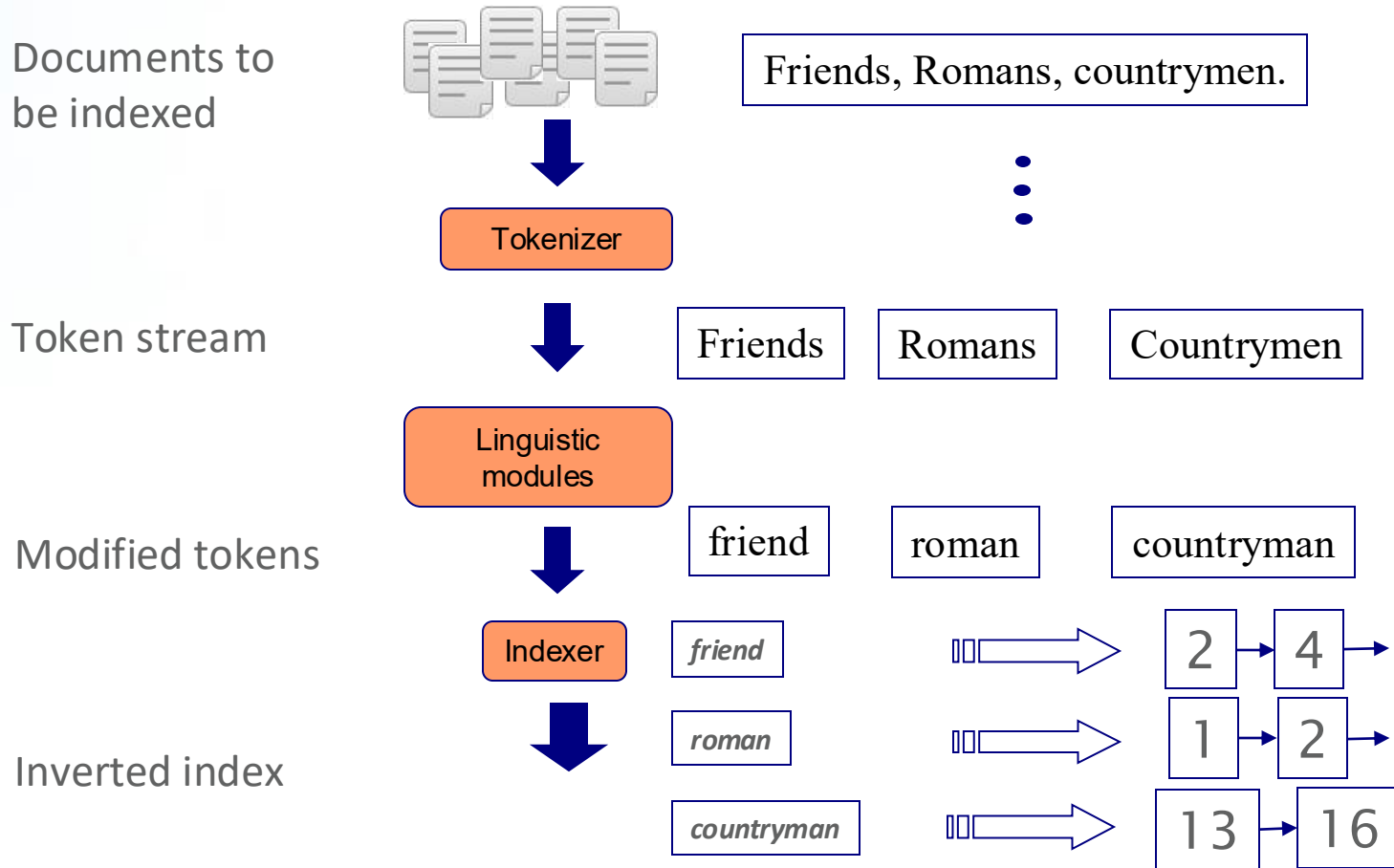
In memory, can use linked lists or variable length arrays

Some tradeoffs in size/ease of insertion



Sorted by docID (more later on why).

Inverted Index Construction



Initial Stages of Text Processing

Tokenization

Cut character sequence into word tokens (=what is a word!)

Deal with *“John’s”, a state-of-the-art solution*

Normalization

Map text (and query) term to same form

You want **U.S.A.** and **USA** to match

Stemming

We may wish to have different forms of a root to match

authorize, authorization

Stop words

We may omit very common words (or not)

the, a, to, of, over, between, his, him

Indexer Steps: Token Sequence

Sequence of (Modified token, Document ID) pairs.

Doc 1

I did enact Julius
Caesar I was killed
i' the Capitol;
Brutus killed me.

Doc 2

So let it be with
Caesar. The noble
Brutus hath told you
Caesar was ambitious



Term	docID
I	1
did	1
enact	1
julius	1
caesar	1
I	1
was	1
killed	1
i'	1
the	1
capitol	1
brutus	1
killed	1
me	1
so	2
let	2
it	2
be	2
with	2
caesar	2
the	2
noble	2
brutus	2
hath	2
told	2
you	2
caesar	2
was	2
ambitious	2

Indexer Steps: Sort

Sort by terms

—And then docID

Core indexing step

Term	docID
I	1
did	1
enact	1
julius	1
caesar	1
I	1
was	1
killed	1
i'	1
the	1
capitol	1
brutus	1
killed	1
me	1
so	2
let	2
it	2
be	2
with	2
caesar	2
the	2
noble	2
brutus	2
hath	2
told	2
you	2
caesar	2
was	2
ambitious	2



Term	docID
ambitious	2
be	2
brutus	1
brutus	2
capitol	1
caesar	1
caesar	2
caesar	2
did	1
enact	1
hath	1
I	1
I	1
i'	1
it	2
julius	1
killed	1
killed	1
let	2
me	1
noble	2
so	2
the	1
the	2
told	2
you	2
was	1
was	2
with	2

Indexer Steps: Dictionary & Postings

Multiple term entries in a single document are merged.

Split into Dictionary and Postings

Doc. frequency information is added.

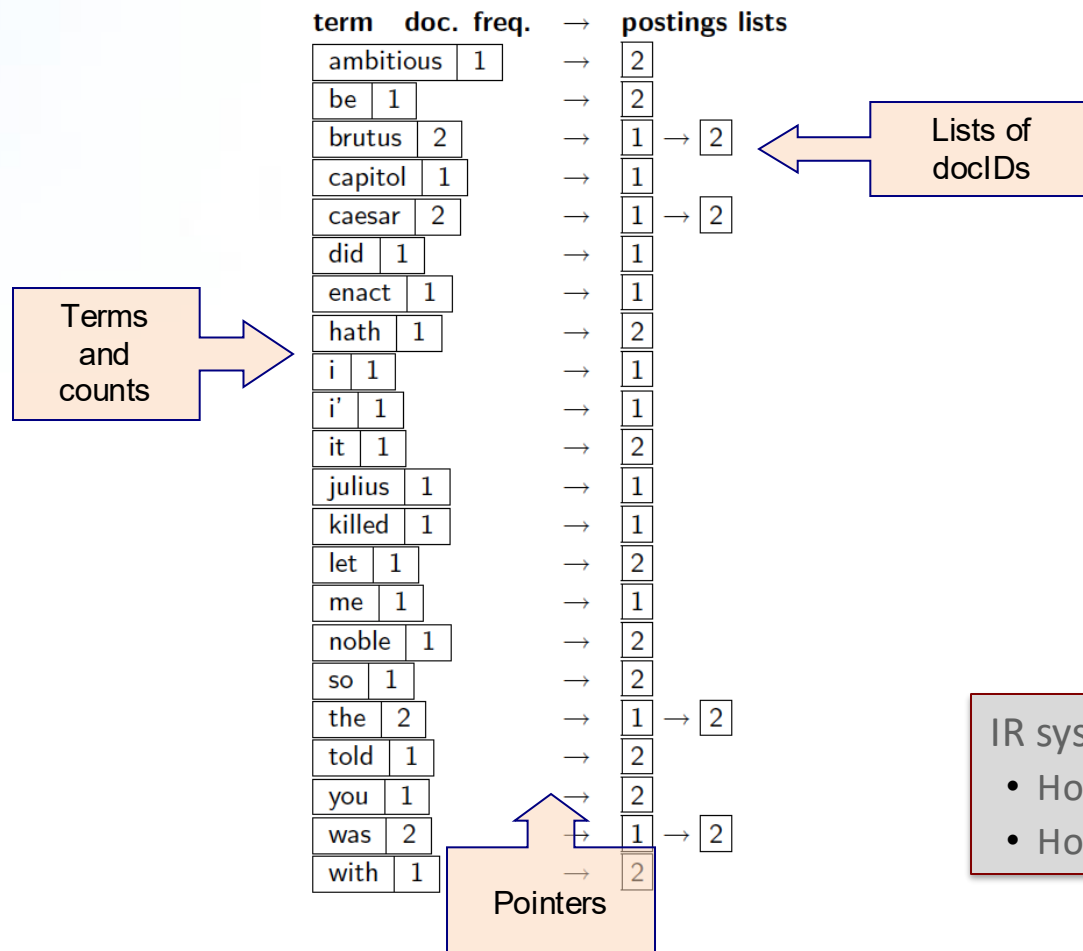
Why frequency?

Term	docID
ambitious	2
be	2
brutus	1
brutus	2
capitol	1
caesar	1
caesar	2
caesar	2
did	1
enact	1
hath	1
i	1
i	1
i'	1
it	2
julius	1
killed	1
killed	1
let	2
me	1
noble	2
so	2
the	1
the	2
told	2
you	2
was	1
was	2
with	2



term	doc. freq.	→	postings lists
ambitious	1	→	2
be	1	→	2
brutus	2	→	1 → 2
capitol	1	→	1
caesar	2	→	1 → 2
did	1	→	1
enact	1	→	1
hath	1	→	2
i	1	→	1
i'	1	→	1
it	1	→	2
julius	1	→	1
killed	1	→	1
let	1	→	2
me	1	→	1
noble	1	→	2
so	1	→	2
the	2	→	1 → 2
told	1	→	2
you	1	→	2
was	2	→	1 → 2
with	1	→	2

Where do we pay in storage?

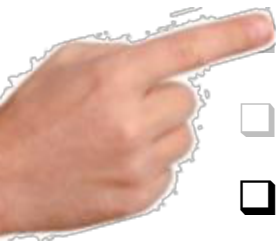


IR system implementation

- How do we index efficiently?
- How much storage do we need?

Introduction to Information Retrieval

In this lecture:

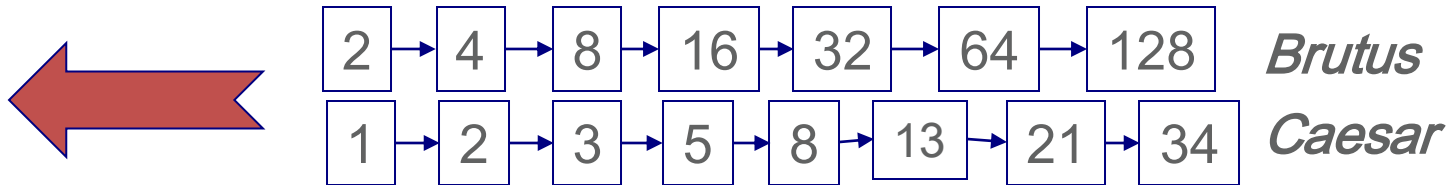
- ☐ Part 1: Information Retrieval Basics
- ☐ Part 2: Term-document incidence matrices
- ☐ Part 3: Inverted Index
-  Part 4: **Query processing with inverted index**
- ☐ Part 5: Phrase queries and positional indexes
- ☒ Part 6: Discussion
- ☒ Part 7: Practice

Query processing: AND

Consider processing the query:

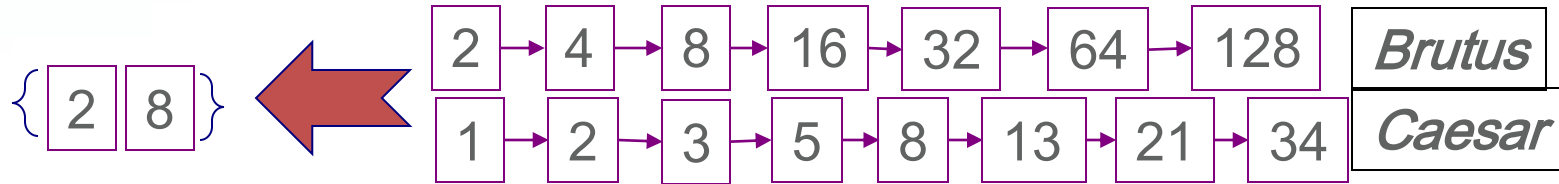
Brutus AND Caesar

- Locate ***Brutus*** in the Dictionary;
 - Retrieve its postings.
- Locate ***Caesar*** in the Dictionary;
 - Retrieve its postings.
- “***Merge***” the two postings (intersect the document sets):



The Merge

Walk through the two postings simultaneously, in time linear in the total number of postings entries



If the list lengths are x and y , the merge takes $O(x+y)$ operations.

Crucial: postings sorted by docID.

Intersecting two postings lists (a “merge” algorithm)

```
INTERSECT( $p_1, p_2$ )
1   $answer \leftarrow \langle \rangle$ 
2  while  $p_1 \neq \text{NIL}$  and  $p_2 \neq \text{NIL}$ 
3  do if  $\text{docID}(p_1) = \text{docID}(p_2)$ 
4      then  $\text{ADD}(answer, \text{docID}(p_1))$ 
5           $p_1 \leftarrow \text{next}(p_1)$ 
6           $p_2 \leftarrow \text{next}(p_2)$ 
7      else if  $\text{docID}(p_1) < \text{docID}(p_2)$ 
8          then  $p_1 \leftarrow \text{next}(p_1)$ 
9          else  $p_2 \leftarrow \text{next}(p_2)$ 
10 return  $answer$ 
```

Introduction to Information Retrieval

In this lecture:

- ☐ Part 1: Information Retrieval Basics
- ☐ Part 2: Term-document incidence matrices
- ☐ Part 3: Inverted Index
- ☐ Part 4: Query processing with inverted index
- ☒ Part 5: **Phrase queries and positional indexes**
- ☐ Part 5: Discussion
- ☐ Part 7: Practice



Phrase Queries

We want to be able to answer queries such as “*stanford university*” as a phrase

Thus the sentence “*I went to university at Stanford*” is not a match.

The concept of phrase queries has proven easily understood by users; one of the few “advanced search” ideas that works

For this, it no longer suffices to store only
<term : docs> entries

First Attempt: Biword Indexes

- Index every consecutive pair of terms in the text as a phrase
- For example the text “*Friends, Romans, Countrymen*” would generate the biwords
 - *friends romans*
 - *romans countrymen*
- Each of these biwords is now a dictionary term
- Two-word phrase query-processing is now immediate.

Longer Phrase Queries

- Longer phrases can be processed by breaking them down
- *stanford university palo alto* can be broken into the Boolean query on biwords:
 - *stanford university* AND *university palo* AND *palo alto*
- Without the docs, we cannot verify that the docs matching the above Boolean query do contain the phrase.

Can have false positives!

Issues for Biword Indexes

- False positives, as noted before
 - Index blowup due to bigger dictionary
 - Infeasible for more than biwords, big even for them
 - Biword indexes are not the standard solution (for all biwords) but can be part of a compound strategy
- ➔ This is not practical/standard solution!

Solution 2: Positional Indexes

In the postings, store, for each ***term*** the position(s) in which tokens of it appear:

<***term***, number of docs containing ***term***;

doc1: position1, position2 ... ;

doc2: position1, position2 ... ;

etc.>

Example: Positional index

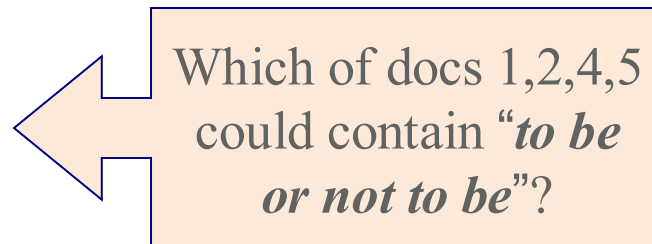
<*be*: 993427;

1: 7, 18, 33, 72, 86, 231;

2: 3, 149;

4: 17, 191, 291, 430, 434;

5: 363, 367, ...>



- For phrase queries, we use a merge algorithm recursively at the document level.
- But we now need to deal with more than just equality.
- **Example:** if we look for “Birzeit University” then if Birzeit in 87 then University should be in 88.

Processing Phrase Queries

Extract inverted index entries for each distinct term: *to*, *be*, *or*, *not*.

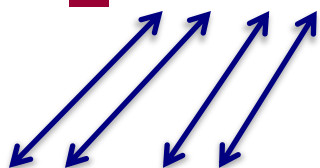
Merge their *doc:position* lists to enumerate all positions with “*to be or not to be*”.

to:

2:1,17,74,222,551; 4:8,16,190,429,433; 7:13,23,191; ...

be:

1:17,19; 4:17,191,291,430,434; 5:14,19,101; ...



Same general method for proximity searches

Positional Index Size

- A positional index expands postings storage *substantially*
 - Even though indices can be compressed.
- Nevertheless, a positional index is now standardly used because of the power and usefulness of phrase and proximity queries ... whether used explicitly or implicitly in a ranking retrieval system.

Rules of thumb

- A positional index is 2–4 as large as a non-positional index
- Positional index size 35–50% of volume of original text

Caveat: all of this holds for “English-like” languages

Combination Schemes (both Solutions)

- These two approaches can be profitably combined
For particular phrases (“*Michael Jackson*”, “*Britney Spears*”) it is inefficient to keep on merging positional postings lists
Even more so for phrases like “*The Who*”
- Williams et al. (2004) evaluate a more sophisticated mixed indexing scheme
A typical web query mixture was executed in $\frac{1}{4}$ of the time of using just a positional index
It required 26% more space than having a positional index alone

Introduction to Information Retrieval

In this lecture:

- ☐ Part 1: Information Retrieval Basics
- ☐ Part 2: Term-document incidence matrices
- ☐ Part 3: Inverted Index
- ☐ Part 4: Query processing with inverted index
- ☐ Part 5: Phrase queries and positional indexes

Part 5: **Discussion**

- ☐ Part 7: Practice



Disussion

Introduction to Information Retrieval

In this lecture:

- ☐ Part 1: Information Retrieval Basics
- ☐ Part 2: Term-document incidence matrices
- ☐ Part 3: Inverted Index
- ☐ Part 4: Query processing with inverted index
- ☐ Part 5: Phrase queries and positional indexes
- ☐ Part 5: Discussion

Part 7: **Practice**



Project (Search Engine)

- Building an Arabic Search Engine based on a positional inverted index. A corpus of at least 10 documents containing at least 10000 words should be used to test this engine.
- The engine should support single, multiple word, as phrase queries.
- The inverted index should take into account Arabic stemming and stop words.
- The results page will list the titles of all relevant documents in a clickable manner.

Deadline: xx/xx/xxxx

References

1. Dan Jurafsky: [From Languages to Information notes](http://web.stanford.edu/class/cs124). <http://web.stanford.edu/class/cs124>
2. Mustafa Jarrar: [The Arabic Ontology - An Arabic Wordnet with Ontologically Clean Content](#). Applied Ontology Journal, 16:1, 1-26. IOS Press. 2021
3. Mustafa Jarrar, Tymaa Hammouda: [Qabas: An Open-Source Arabic Lexicographic Database](#). In Proceedings of IREC-COUNG 2024, pages 13363–13370, Torino, Italia. ELRA and ICCL
4. Mustafa Jarrar, Sanad Malaysha, Tymaa Hammouda, Mohammed Khalilia: [SALMA: Arabic Sense-Annotated Corpus and WSD Benchmarks](#). Proceedings the 1st ArabicNLP, Part of the ACL 2023. ACL.
5. Sana Ghanem, Mustafa Jarrar, Radi Jarrar, Ibrahim Bounhas: [A Benchmark and Scoring Algorithm for Enriching Arabic Synonyms](#). In Proceedings of GWC2023, (pp.274-283). Spain, 2023
6. Sanad Malaysha, Mustafa Jarrar, Mohammed Khalilia: [Context-Gloss Augmentation for Improving Arabic Target Sense Verification](#). In Proceedings of GWC2023, (pp.274-283). Spain, 2023
7. Moustafa Al-Hajj, Mustafa Jarrar: [ArabGlossBERT: Fine-Tuning BERT on Context-Gloss Pairs for WSD](#). In Proceedings of the International Conference on Recent Advances in Natural Language Processing (RANLP 2021).
8. Moustafa Al-Hajj, Mustafa Jarrar: [LU-BZU at SemEval-2021 Task 2: Word2Vec and Lemma2Vec performance in Arabic Word-in-Context disambiguation](#). In Proceedings of the Fifteenth Workshop on Semantic Evaluation (SemEval2021) Task 2: Multilingual and Cross-lingual Word-in-Context Disambiguation (MCL-WiC). PP 748–755, Association for Computational Linguistics. 2021
9. Mustafa Jarrar, Eman Karajah, Muhammad Khalifa, Khaled Shaalan: [Extracting Synonyms from Bilingual Dictionaries](#). The 11th International Global Wordnet Conference (GWC2021), Global Wordnet Association. (pp. 215-222). Pretoria, South Africa, 2021
10. Mustafa Jarrar, Hamzeh Amayreh: [An Arabic-Multilingual Database with a Lexicographic Search Engine](#). The 24th International Conference on Applications of Natural Language to Information Systems (NLDB 2019). Pages(234246). LNCS 11608, Springer. 2019
11. Mustafa Jarrar, Hamzeh Amayreh, John P. McCrae: [Representing Arabic Lexicons in Lemon - a Preliminary Study](#). The 2nd Conference on Language, Data and Knowledge (LDK 2019). Pages(2933). CEUR, Volume 2402. ISSN:1613-0073. Leipzig, Germany. 2019
12. Diana Alhafi, Anton Deik, Mustafa Jarrar: [Usability Evaluation of Lexicographic Services](#). The 16th IEEE/ACS International Conference on Computer Systems and Applications (AICCSA). Pages(1-7). IEEE. 2019
13. Mustafa Jarrar, Fadi Zaraket, Rami Asia, Hamzeh Amayreh: [Diacritic-Based Matching of Arabic Words](#). ACM Asian and Low-Resource Language Information Processing. Volume 18, No 2, Pages (10:11021), ACM, ISSN:2375-4699. December, 2018
14. Mustafa Jarrar, Werner Ceusters: [Classifying Processes and Basic Formal Ontology](#). Proceedings of the 8th International Conference on Biomedical Ontology (ICBO 2017), Newcastle, UK. 2017
15. Mustafa Jarrar: [Building a Formal Arabic Ontology \(Invited Paper\)](#). In proceedings of the [Experts Meeting on Arabic Ontologies and Semantic Networks](#). Aleco, Arab League. Tunis, July 26-28, 2011.
16. Mustafa Jarrar: [Towards the notion of gloss, and the adoption of linguistic resources in formal ontology engineering](#). In proceedings of the 15th International World Wide Web Conference (WWW2006). Edinburgh, Scotland. Pages 497-503. ACM Press. ISBN: 1595933239. May 2006.
17. Mustafa Jarrar, Anton Deik, Bilal Faraj: [Ontology-based Data and Process Governance Framework -The Case of e-Government Interoperability in Palestine](#). Proceedings of the IFIP International Symposium on Data-Driven Process Discovery and Analysis (SIMPDA'11). Pages(83-98). 2011.
18. Mustafa Jarrar and Robert Meersman: [Ontology Engineering -The DOGMA Approach](#). Book Chapter in "Advances in Web Semantics I". Chapter 3. Pages 7-34. LNCS 4891, Springer. (2008).
19. Mustafa Jarrar: [Tutorial on Arabic Ontology Engineering](#). The ACS/IEEE International Conference on Computer Systems and Applications. Tunis, 2017
20. Mustafa Jarrar, Maria Keet, and Paolo Dongilli: [Multilingual verbalization of ORM conceptual models and axiomatized ontologies](#). Technical report. STARLab, Vrije Universiteit Brussel, February 2006.
21. Mustafa Jarrar: [Mapping ORM into the SHOIN/OWL Description Logic: Towards a Methodological and Expressive Graphical Notation for Ontology Engineering](#). In OTM 2007 workshops: Proceedings of the International Workshop on Object-Role Modeling (ORM'07). Pages (729-741). LNCS 4805, Springer. ISBN: 9783540768890. Portugal. November, 2007
22. Tymaa Hammouda, Mustafa Jarrar, Mohammed Khalilia: [SinaTools: Open Source Toolkit for Arabic Natural Language Understanding](#). In Proceedings of the 2024 AI in Computational Linguistics (ACLING 2024), Procedia Computer Science, Dubai. ELSEVIER.
23. Mustafa Jarrar, Diyam Akra, Tymaa Hammouda: [ALMA: Fast Lemmatizer and POS Tagger for Arabic](#). Proceedings of the 2024 AI in Computational Linguistics (ACLING), Procedia Computer Science. ELSEVIER.
24. Alaa Aljabari, Lina Duaiabes, Mustafa Jarrar, Mohammed Khalilia: [Event-Arguments Extraction Corpus and Modeling using BERT for Arabic](#). In Proceedings of the Second Arabic Natural Language Processing Conference (ArabicNLP 2024), Bangkok, Thailand. Association for Computational Linguistics.
25. Mohammed Khalilia, Sanad Malaysha, Reem Suwaileh, Mustafa Jarrar, Alaa Aljabari, Tamer Elsayed, Imed Zitouni: [ArabicNLU 2024: The First Arabic Natural Language Understanding Shared Task](#). In Proceedings of the Second Arabic Natural Language Processing Conference (ArabicNLP 2024), Bangkok, Thailand. Association for Computational Linguistics.
26. Mustafa Jarrar, Nagham Hamad, Mohammed Khalilia, Bashar Talafha, AbdelRahim Elmadany, Muhammad Abdul-Mageed: [WojoodNER 2024: The Second Arabic Named Entity Recognition Shared Task](#). In Proceedings of the Second Arabic Natural Language Processing Conference (ArabicNLP 2024), Bangkok, Thailand. Association for Computational Linguistics.
27. Amal Nayouf, Tymaa Hammouda, Mustafa Jarrar, Fadi Zaraket, Mohamad-Bassam Kurd: [Nābra: Syrian Arabic Dialects with Morphological Annotations](#). In Proceedings of the 1st Arabic Natural Language Processing Conference (ArabicNLP), Part of the TKM24 2023. ACL.
28. Mustafa Jarrar, Ahmet Birim, Mohammed Khalilia, Mustafa Erden, and Sana Ghanem: [ArBanking77: Intent Detection Neural Model and a New Dataset in Modern and Dialectal Arabic](#). In Proceedings of the 1st Arabic Natural Language Processing Conference (ArabicNLP), Part of the TKM24 2023. ACL.
29. Mustafa Jarrar, Fadi Zaraket, Tymaa Hammouda, Daanish Masood, Martin Waehlich: [Lisan: Yemeni, Iraqi, Libyan, and Sudanese Arabic Dialect Corpora with Morphological Annotations](#). The 20th ACS/IEEE International Conference on Computer Systems and Applications (AICCSA). Pages(1-7). IEEE. Egypt. DOI 10.1109/AICCSA59173.2023.10479250. 2023
30. Karim El Haff, Mustafa Jarrar, Tymaa Hammouda, Fadi Zaraket: [Curras + Baladi: Towards a Levantine Corpus](#). Proceedings of the International Conference on Language Resources and Evaluation (LREC), 2022